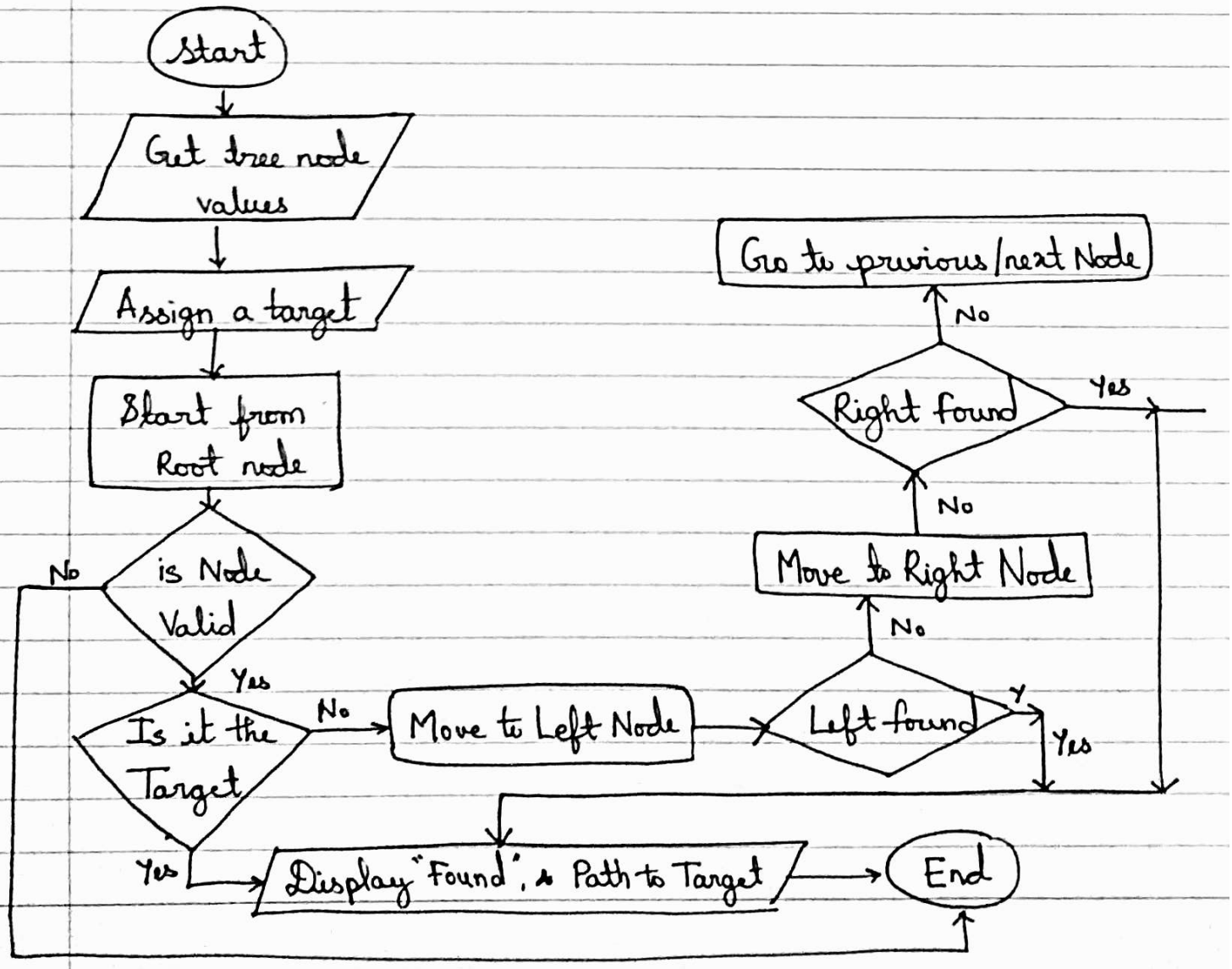
Think like a Coder – The Gauntlet

Algorithm:

1. Start
2. Get the tree node values
3. Assign a random node as target
4. Start from node 1 ie first node / Root node
5. Check if the node is valid
6. If node is Target, display Found, go to step 10
7. Else if move to ~~node~~ left node, go to step 5
8. Else if move to right node, go to step 5
9. ~~If~~ ~~Not~~ Else go back to previous node.
10. Display the path to the Target
11. End

Flowchart:



Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
char path[15];
int tree[15];
int target, length = 0;
int findpath(int node) {
  if (node >= 15 || tree[node]==0)
    return 0;
  if (node == target)
    return 1;
  path[length] = 'L';
  length++;
  if(findpath(2*node+1))
    return 1;
  length--;
  path[length] = 'R'
  length++
  if (findpath(2*node+2))
    return 1;
  length--;
  return 0;
}
int main() {
  int i;
  int found = 0;
  printf("Enter Tree node values:");
  printf("\n 1->Open node 0-> Closed
    node \n");
  for(i=0; i<15; i++) {
    printf("Node %d: ", i);
    scanf("%d", &tree[i]);
  }
  target = rand()%15;
  while (!found) {
    if (target == 0 || tree[target]
        == 0) continue;
    length = 0;
    if (findpath(0))
      found = 1;
  }
  printf("Path: Start ");
  for(i=0, i<length; i++) {
    if (path[i] == 'L')
      printf("-> Left ");
    else
      printf("-> Right");
  }
  printf("--> Target \n");
  return 0;
}
```